TÀI LIỆU HỌC TẬP — DÀNH CHO SINH VIÊN

BUỔI 4: BÀI TOÁN TÌM KIẾM MÙ (BFS, DFS)

Giải quyết bài toán bằng không gian trạng thái

Họ và tên: Nguyễn Quách Phú Tài	MSSV: 123000609	Nhóm: ……	Ngày: 6/10/2026

HƯỚNG DẪN SỬ DỤNG TÀI LIỆU

- Tài liệu này đã có sẵn toàn bộ nội dung/code cho từng hoạt động — bạn không cần tự viết code từ đầu. Quy trình cho mỗi hoạt động là: 📋 Copy code có sẵn → ▶️ Paste vào Colab và chạy → 💭 Suy nghĩ/quan sát → 🔧 Cải tiến (tự thử thay đổi) → 🌾 Thu hoạch (ghi lại điều học được)
- Ở bước 🔧 Cải tiến, hãy dán phần code bạn đã tự chỉnh sửa vào khung viền vàng đứt nét; ở bước 🌾 Thu hoạch, hãy chụp màn hình (screenshot) kết quả (kể cả hình mê cung) và dán vào khung viền xám đứt nét (🖼️)
- Ghi câu trả lời/suy nghĩ của bạn vào các dòng kẻ chấm — không cần viết dài, chỉ cần thật (đây không phải bài kiểm tra đúng/sai)
- Cuối Tiết 4, hoàn thành phần "BÁO CÁO TRẢI NGHIỆM" ở cuối tài liệu này
- Trước khi rời lớp: vào File → Download → PDF Document để xuất toàn bộ tài liệu (đã có code cải tiến + ảnh + ghi chú của bạn) thành file PDF và nộp theo hướng dẫn của giảng viên

MỤC TIÊU BUỔI HỌC

Sau buổi học này, bạn sẽ có khả năng:

- Giải thích được không gian trạng thái và cây tìm kiếm là gì
- Cài đặt được BFS và DFS để giải bài toán tìm đường
- Phân biệt được cơ chế Queue (FIFO) của BFS và Stack (LIFO) của DFS
- Giải thích được vì sao BFS luôn tìm đường ngắn nhất còn DFS thì không
- So sánh được tốc độ và số ô duyệt của BFS/DFS trên cùng một bài toán

TIẾT 1 — TRẢI NGHIỆM: CÙNG MỘT MÊ CUNG, HAI CÁCH GIẢI

Mở notebook "Demo_Buoi4.ipynb" (đã có sẵn toàn bộ code bên dưới). Với mỗi demo, bạn chỉ cần: copy code → paste vào một cell Colab → nhấn Shift+Enter để chạy. Sau đó suy nghĩ, thử cải tiến, và ghi lại thu hoạch.

Demo 1: Giải mê cung bằng BFS

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
from collections import deque
import matplotlib.pyplot as plt
import numpy as np

maze = [
    [0,0,0,0,0],
    [1,1,0,1,0],
    [0,0,0,1,0],
    [0,1,1,1,0],
    [0,0,0,0,0],
]
```

```
start, goal = (0,0), (4,4)

def bfs(maze, start, goal):
    rows, cols = len(maze), len(maze[0])
    queue = deque([start])
    visited = {start: None}
    while queue:
        current = queue.popleft()
        if current == goal:
            break
        r, c = current
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = r+dr, c+dc
               if 0 <= nr < rows and 0 <= nc < cols and maze[nr][nc]==0 and
(nr,nc) not in visited:
                visited[(nr,nc)] = current
                queue.append((nr,nc))
    path, node = [], goal
    while node is not None:
        path.append(node)
        node = visited.get(node)
    path.reverse()
    return path, len(visited)

path_bfs, so_node_bfs = bfs(maze, start, goal)
print("Đường đi BFS:", path_bfs)
print("Số ô đã duyệt:", so_node_bfs)

def ve_mien(maze, path, title):
    plt.imshow(np.array(maze), cmap="binary")
    if path:
        xs = [c for r, c in path]
        ys = [r for r, c in path]
        plt.plot(xs, ys, color="red", linewidth=2, marker="o")
    plt.title(title)
    plt.show()

ve_mien(maze, path_bfs, "Đường đi BFS")
```

💭 **Suy nghĩ: Đường đi BFS tìm được có phải là đường ngắn nhất không? Vì sao bạn nghĩ vậy?**

Có. Đường BFS dài 8 bước: (0,0) → (0,1) → (0,2) → (0,3) → (0,4) → (1,4) → (2,4) → (3,4) → (4,4), duyệt 16 ô.

Mình nghĩ đây là đường ngắn nhất vì BFS lấy ô bằng hàng đợi FIFO. Ô gần start hơn luôn được xét trước ô xa hơn. Mỗi nước đi tốn đúng 1 bước, nên lần đầu chạm goal chính là đường ít bước nhất. Các đường khác, nếu có, chỉ có thể dài hơn hoặc bằng.

🔧 Cải tiến: Hãy đổi start hoặc goal sang một ô trống khác trong mê cung, chạy lại và xem đường đi mới:

```
from collections import deque
import matplotlib.pyplot as plt
import numpy as np

maze = [
  [0, 0, 0, 0, 0],
  [1, 1, 0, 1, 0],
  [0, 0, 0, 1, 0],
  [0, 1, 1, 1, 0],
  [0, 0, 0, 0, 0],
]
# Cải tiến: giữ start, đổi goal sang ô trống (2, 0)
```

```
start, goal = (0, 0), (2, 0)

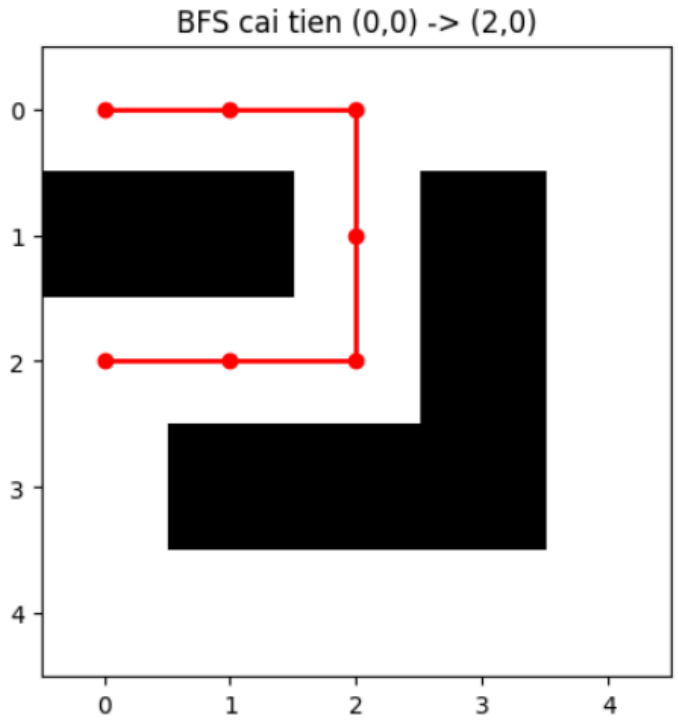
def bfs(maze, start, goal):
    rows, cols = len(maze), len(maze[0])
    queue = deque([start])
    visited = {start: None}
    while queue:
        current = queue.popleft()
        if current == goal:
            break
        r, c = current
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nr, nc = r + dr, c + dc
            if (0 <= nr < rows and 0 <= nc < cols
                    and maze[nr][nc] == 0 and (nr, nc) not in visited):
                visited[(nr, nc)] = current
                queue.append((nr, nc))
    path, node = [], goal
    while node is not None:
        path.append(node)
        node = visited.get(node)
    path.reverse()
    return path, len(visited)

def ve_mien(maze, path, title):
    plt.imshow(np.array(maze), cmap="binary")
    if path:
        xs = [c for r, c in path]
        ys = [r for r, c in path]
        plt.plot(xs, ys, color="red", linewidth=2, marker="o")
    plt.title(title)
    plt.show()

path_bfs, so_node_bfs = bfs(maze, start, goal)
print("Đường đi BFS:", path_bfs)
print("Số ô đã duyệt:", so_node_bfs)
print("Số bước:", len(path_bfs) - 1)
ve_mien(maze, path_bfs, "BFS cải tiến (0,0) -> (2,0)")
```

🌾 Thu hoạch: chụp màn hình đường đi BFS sau khi cải tiến:

```
Duong di BFS: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0)]
So o da duyet: 11
So buoc: 6
```



Demo 2: Giải cùng mê cung bằng DFS

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
def dfs(maze, start, goal):
    rows, cols = len(maze), len(maze[0])
    stack = [start]
    visited = {start: None}
    while stack:
        current = stack.pop()
        if current == goal:
            break
        r, c = current
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = r+dr, c+dc
               if 0 <= nr < rows and 0 <= nc < cols and maze[nr][nc]==0 and
(nr,nc) not in visited:
                visited[(nr,nc)] = current
                stack.append((nr,nc))
    path, node = [], goal
    while node is not None:
        path.append(node)
        node = visited.get(node)
    path.reverse()
    return path, len(visited)

path_dfs, so_node_dfs = dfs(maze, start, goal)
print("Đường đi DFS:", path_dfs)
print("Số ô đã duyệt:", so_node_dfs)
ve_mien(maze, path_dfs, "Đường đi DFS")
```

💭 **Suy nghĩ: So với BFS, đường đi DFS có ngắn hơn/dài hơn/bằng không? Số ô đã duyệt của DFS so với BFS thế nào?**

Trên mê cung gốc, đường DFS bằng đường BFS: cùng 8 bước, cùng dãy ô. Số ô duyệt thì DFS ít hơn, 10 so với 16.

Trùng đường là do thứ tự đẩy stack. Ô bên phải được append sau cùng nên bị pop ra trước, mà hành lang bên phải tình cờ là đường ngắn. DFS dừng ngay khi gặp goal, nên không duyệt nốt các ô ở cột trái.

Không phải lúc nào cũng bằng. Đổi goal sang (2,0) thì DFS dài 14 bước và duyệt 16 ô, BFS chỉ 6 bước và 11 ô.

🔧 Cải tiến: Hãy dùng cùng start/goal mà bạn đã đổi ở Demo 1, chạy DFS và so sánh 2 kết quả:

```
from collections import deque
import matplotlib.pyplot as plt
import numpy as np

maze = [
    [0, 0, 0, 0, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 1, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0],
]
# Cùng start/goal đã đổi ở Demo 1
start, goal = (0, 0), (2, 0)

def dfs(maze, start, goal):
    rows, cols = len(maze), len(maze[0])
    stack = [start]
    visited = {start: None}
    while stack:
        current = stack.pop()
        if current == goal:
            break
        r, c = current
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nr, nc = r + dr, c + dc
            if (0 <= nr < rows and 0 <= nc < cols
                    and maze[nr][nc] == 0 and (nr, nc) not in visited):
                visited[(nr, nc)] = current
                stack.append((nr, nc))
    path, node = [], goal
    while node is not None:
        path.append(node)
        node = visited.get(node)
    path.reverse()
    return path, len(visited)

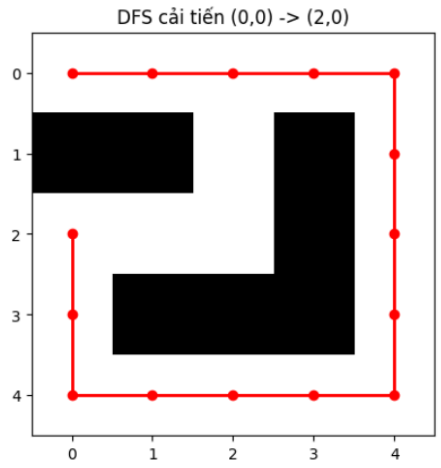
def ve_mien(maze, path, title):
    plt.imshow(np.array(maze), cmap="binary")
    if path:
        xs = [c for r, c in path]
        ys = [r for r, c in path]
        plt.plot(xs, ys, color="red", linewidth=2, marker="o")
    plt.title(title)
    plt.show()

path_dfs, so_node_dfs = dfs(maze, start, goal)
print("Đường đi DFS:", path_dfs)
```

```
print("Số ô đã duyệt:", so_node_dfs)
print("Số bước:", len(path_dfs) - 1)
ve_mien(maze, path_dfs, "DFS cải tiến (0,0) -> (2,0)")
```

🌾 Thu hoạch: chụp màn hình đường đi DFS để so sánh với BFS:

```
Đường đi DFS: [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (3, 4), (4, 4), (4, 3), (4, 2), (4, 1), (4, 0), (3, 0), (2, 0)]
Số ô đã duyệt: 16
Số bước: 14
```



Câu hỏi thảo luận: Vì sao 2 đường đi khác nhau dù cùng xuất phát và cùng đích? Điều đó có luôn đúng với mọi mê cung không?

Hai đường khác nhau vì cách lấy ô kế tiếp khác nhau, không phải vì mê cung đổi.

BFS dùng hàng đợi: ô nào vào trước thì được xét trước, nên nó đi hết các ô cách start đúng k bước rồi mới sang k+1. Lần đầu chạm đích chính là đường ít bước nhất. Với `(0,0) → (2,0)`, đường đó dài 6 bước: `(0,0) → (0,1) → (0,2) → (1,2) → (2,2) → (2,1) → (2,0)`.

DFS dùng ngăn xếp: ô vừa đẩy vào được lấy ra ngay. Trong code, láng giềng được xét theo thứ tự trên, dưới, trái, phải, nên ô bên phải vào stack sau cùng và bị pop trước. DFS vì vậy lao hết hàng 0 sang `(0,4)`, đi xuống cột phải, vòng dưới đáy rồi mới tới `(2,0)` — 14 bước.

Điều đó không đúng với mọi mê cung. Nếu chỉ có một đường, hai thuật toán trùng nhau. Mê cung gốc `(0,0) → (4,4)` cũng trùng 8 bước, vì nhánh DFS cắm vào trước tình cờ là hành lang ngắn bên phải. Chỉ khi nhánh được xét trước không phải đường ngắn, như goal `(2,0)`, thì DFS mới dài hơn BFS.

TIẾT 2 — KHÁI QUÁT HÓA: GHI CHÚ LÝ THUYẾT

Vừa nghe giảng vừa điền vào các mục dưới đây bằng lời của chính bạn (không chép nguyên văn slide):

Không gian trạng thái (state space) và cây tìm kiếm (search tree) là gì?

Không gian trạng thái là tập mọi tình huống bài toán có thể rơi vào, cùng các nước đi nối chúng. Với mê cung, mỗi ô trống là một trạng thái; đi lên, xuống, trái, phải là một cạnh, miễn là không đụng tường và không ra ngoài biên. Cây tìm kiếm là cách thuật toán mở các trạng thái đó từ start: nút gốc là start, con của một nút là các ô kề chưa xét. Cây này không phải bản đồ. Nhờ tập visited, mỗi ô chỉ được ghi nhận một lần, theo người cha đầu tiên dẫn tới nó.

BFS dùng cấu trúc dữ liệu nào (Queue/Stack)? DFS dùng cấu trúc dữ liệu nào? Vì sao chỉ khác cách lấy phần tử mà hành vi lại khác hẳn nhau?

BFS dùng Queue, lấy bằng popleft: vào trước thì ra trước. DFS dùng Stack, lấy bằng pop: vào sau thì ra trước. Phần còn lại của hai hàm gần như giống nhau, nhưng thứ tự mở nút đổi hết. Queue làm BFS xong

một lớp khoảng cách rồi mới sang lớp sau. Stack làm DFS đi hết một ngả, các ngả còn lại nằm chờ bên dưới. Đường dựng lại theo cha cũng vì thế mà khác nhau.

Tính 'đầy đủ' (completeness) và 'tối ưu' (optimality) nghĩa là gì? BFS/DFS thuật toán nào tối ưu, thuật toán nào không?

Đầy đủ nghĩa là nếu có đường thì thuật toán tìm ra, không bỏ sót. Tối ưu nghĩa là đường tìm được là đường tốt nhất theo tiêu chí đang dùng — ở mê cung này là ít bước nhất. Trên đồ thị hữu hạn và có visited, cả BFS lẫn DFS đều đầy đủ. Chỉ BFS tối ưu khi mọi cạnh cùng chi phí. DFS đầy đủ nhưng không tối ưu: nó trả về nhánh chạm đích trước, không phải nhánh ngắn nhất.

Khi nào bạn sẽ chọn dùng BFS? Khi nào bạn sẽ chọn dùng DFS?

Chọn BFS khi cần đường ngắn nhất hoặc ít bước nhảy: mê cung, traceroute, crawler muốn trang gần gốc trước. Chọn DFS khi chỉ cần một đường bất kỳ, khi cần thứ tự cài đặt hay sắp xếp tô-pô, khi cần phát hiện vòng, hoặc khi chỉ muốn giữ một nhánh. Trên cây thì DFS cũng ra đường ngắn, vì chỉ có một đường.

TIẾT 3 — ÁP DỤNG CÓ HƯỚNG DẪN: BFS/DFS TRÊN ĐỒ THỊ BẢN ĐỒ

📋 Nội dung có sẵn — tìm đường trên đồ thị các tỉnh thành bằng BFS. Copy và chạy:

```
from collections import deque

do_thi = {
    "HCM": ["BinhDuong", "DongNai", "LongAn"],
    "BinhDuong": ["HCM", "BinhPhuoc"],
    "DongNai": ["HCM", "BaRiaVungTau"],
    "LongAn": ["HCM", "TienGiang"],
    "BinhPhuoc": ["BinhDuong"],
    "BaRiaVungTau": ["DongNai"],
    "TienGiang": ["LongAn", "CanTho"],
    "CanTho": ["TienGiang"],
}

def bfs_do_thi(do_thi, start, goal):
    queue = deque([[start]])
    visited = {start}
    while queue:
        path = queue.popleft()
        node = path[-1]
        if node == goal:
            return path
        for neighbor in do_thi.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(path + [neighbor])
    return None

duong_di = bfs_do_thi(do_thi, "HCM", "CanTho")
print("Đường đi (BFS) từ HCM đến Cần Thơ:", duong_di)
```

📋 Nội dung có sẵn — cùng bài toán nhưng giải bằng DFS. Copy và chạy:

```
def dfs_do_thi(do_thi, start, goal):
    stack = [[start]]
    visited = {start}
    while stack:
        path = stack.pop()
        node = path[-1]
        if node == goal:
            return path
```

```
        for neighbor in do_thi.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                stack.append(path + [neighbor])
    return None

duong_di_dfs = dfs_do_thi(do_thi, "HCM", "CanTho")
print("Đường đi (DFS) từ HCM đến Cần Thơ:", duong_di_dfs)
```

💭 **Suy nghĩ: 2 đường đi BFS và DFS từ HCM đến Cần Thơ có giống nhau không?**

Giống nhau: cả hai đều ra `HCM → LongAn → TienGiang → CanTho`, 3 bước.

Giống không phải vì DFS luôn tìm được đường ngắn nhất. `LongAn` nằm cuối danh sách kề của `HCM`, nên được đẩy vào stack sau cùng và bị `pop` ra trước. DFS cắm đúng nhánh ngắn nhất rồi gặp Cần Thơ, dừng. Nếu `LongAn` không nằm cuối danh sách, DFS có thể đi Bình Dương hoặc Đồng Nai trước và ra đường khác.

🔧 Cải tiến: (1) Thêm ít nhất 2 tỉnh thành mới vào dictionary do_thi (có kết nối với các tỉnh đã có); (2) đổi điểm bắt đầu/kết thúc và chạy lại cả BFS và DFS:

```
from collections import deque

# Đồ thị gốc + 2 tỉnh mới: Tây Ninh, Vĩnh Long
do_thi = {
    "HCM": ["BinhDuong", "DongNai", "LongAn"],
    "BinhDuong": ["HCM", "BinhPhuoc", "TayNinh"],
    "DongNai": ["HCM", "BaRiaVungTau"],
    "LongAn": ["HCM", "TienGiang", "TayNinh"],
    "BinhPhuoc": ["BinhDuong"],
    "BaRiaVungTau": ["DongNai"],
    "TienGiang": ["LongAn", "CanTho", "VinhLong"],
    "CanTho": ["TienGiang", "VinhLong"],
    "TayNinh": ["BinhDuong", "LongAn"],
    "VinhLong": ["TienGiang", "CanTho"],
}

def bfs_do_thi(do_thi, start, goal):
    queue = deque([[start]])
    visited = {start}
    while queue:
        path = queue.popleft()
        node = path[-1]
        if node == goal:
            return path
        for neighbor in do_thi.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(path + [neighbor])
    return None

def dfs_do_thi(do_thi, start, goal):
    stack = [[start]]
    visited = {start}
    while stack:
        path = stack.pop()
        node = path[-1]
        if node == goal:
            return path
        for neighbor in do_thi.get(node, []):
```

```
        if neighbor not in visited:
            visited.add(neighbor)
            stack.append(path + [neighbor])
    return None

print("BFS HCM -> CanTho:", bfs_do_thi(do_thi, "HCM", "CanTho"))
print("DFS HCM -> CanTho:", dfs_do_thi(do_thi, "HCM", "CanTho"))
print("BFS BRVT -> TayNinh:", bfs_do_thi(do_thi, "BaRiaVungTau", "TayNinh"))
print("DFS BRVT -> TayNinh:", dfs_do_thi(do_thi, "BaRiaVungTau", "TayNinh"))
print("BFS CanTho -> BinhPhuoc:", bfs_do_thi(do_thi, "CanTho", "BinhPhuoc"))
print("DFS CanTho -> BinhPhuoc:", dfs_do_thi(do_thi, "CanTho", "BinhPhuoc"))
print("BFS VinhLong -> BRVT:", bfs_do_thi(do_thi, "VinhLong", "BaRiaVungTau"))
print("DFS VinhLong -> BRVT:", dfs_do_thi(do_thi, "VinhLong", "BaRiaVungTau"))
```

🌾 Thu hoạch: chụp màn hình kết quả 2 đường đi sau khi cải tiến:

```
print("BFS VinhLong -> BRVT:", bfs_do_thi(do_thi, "VinhLong", "BaRiaVungTau"))
print("DFS VinhLong -> BRVT:", dfs_do_thi(do_thi, "VinhLong", "BaRiaVungTau"))
```

```
BFS HCM -> CanTho: ['HCM', 'LongAn', 'TienGiang', 'CanTho']
DFS HCM -> CanTho: ['HCM', 'LongAn', 'TienGiang', 'CanTho']
BFS BRVT -> TayNinh: ['BaRiaVungTau', 'DongNai', 'HCM', 'BinhDuong', 'TayNinh']
DFS BRVT -> TayNinh: ['BaRiaVungTau', 'DongNai', 'HCM', 'LongAn', 'TayNinh']
BFS CanTho -> BinhPhuoc: ['CanTho', 'TienGiang', 'LongAn', 'HCM', 'BinhDuong', 'BinhPhuoc']
DFS CanTho -> BinhPhuoc: ['CanTho', 'TienGiang', 'LongAn', 'TayNinh', 'BinhDuong', 'BinhPhuoc']
BFS VinhLong -> BRVT: ['VinhLong', 'TienGiang', 'LongAn', 'HCM', 'DongNai', 'BaRiaVungTau']
DFS VinhLong -> BRVT: ['VinhLong', 'TienGiang', 'LongAn', 'HCM', 'DongNai', 'BaRiaVungTau']
```

TIẾT 4 — ÁP DỤNG MỞ RỘNG: SO SÁNH HIỆU NĂNG BFS VS DFS

Làm việc theo nhóm 2–3 người.

📋 Nội dung có sẵn — copy và chạy code so sánh tốc độ/số ô duyệt trên một mê cung lớn hơn (tái sử dụng hàm bfs/dfs ở Tiết 1):

```
import time

maze_lon = [
    [0,0,0,0,0,0,0,0],
    [0,1,1,1,0,1,1,0],
    [0,1,0,0,0,0,1,0],
    [0,1,0,1,1,0,1,0],
    [0,0,0,1,0,0,0,0],
    [1,1,0,1,0,1,1,0],
    [0,0,0,0,0,1,0,0],
    [0,1,1,1,0,0,0,0],
]
start, goal = (0,0), (7,7)

t0 = time.time()
path_bfs, so_node_bfs = bfs(maze_lon, start, goal)
thoi_gian_bfs = time.time() - t0

t0 = time.time()
path_dfs, so_node_dfs = dfs(maze_lon, start, goal)
thoi_gian_dfs = time.time() - t0

print("BFS - Số ô duyệt:", so_node_bfs, "| Thời gian:", thoi_gian_bfs, "| Độ
dài đường đi:", len(path_bfs))
```

```
print("DFS - Số ô duyệt:", so_node_dfs, "| Thời gian:", thoi_gian_dfs, "| Độ
dài đường đi:", len(path_dfs))

ve_mien(maze_lon, path_bfs, "BFS trên mê cung lớn")
ve_mien(maze_lon, path_dfs, "DFS trên mê cung lớn")
```

☐ Chạy code trên và so sánh số ô duyệt + độ dài đường đi của BFS và DFS

🔧 Cải tiến: Cả nhóm cùng thiết kế một mê cung riêng (kích thước tự chọn, VD 10x10) với các bức tường do nhóm tự vẽ (giá trị 1), chạy lại toàn bộ so sánh BFS/DFS trên mê cung của nhóm:

```
from collections import deque
import time
import matplotlib.pyplot as plt
import numpy as np

def bfs(maze, start, goal):
    rows, cols = len(maze), len(maze[0])
    queue = deque([start])
    visited = {start: None}
    while queue:
        current = queue.popleft()
        if current == goal:
            break
        r, c = current
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nr, nc = r + dr, c + dc
            if (0 <= nr < rows and 0 <= nc < cols
                    and maze[nr][nc] == 0 and (nr, nc) not in visited):
                visited[(nr, nc)] = current
                queue.append((nr, nc))
    path, node = [], goal
    while node is not None:
        path.append(node)
        node = visited.get(node)
    path.reverse()
    return path, len(visited)

def dfs(maze, start, goal):
    rows, cols = len(maze), len(maze[0])
    stack = [start]
    visited = {start: None}
    while stack:
        current = stack.pop()
        if current == goal:
            break
        r, c = current
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nr, nc = r + dr, c + dc
            if (0 <= nr < rows and 0 <= nc < cols
                    and maze[nr][nc] == 0 and (nr, nc) not in visited):
                visited[(nr, nc)] = current
                stack.append((nr, nc))
    path, node = [], goal
    while node is not None:
        path.append(node)
        node = visited.get(node)
    path.reverse()
    return path, len(visited)
```

```
def ve_mien(maze, path, title):
    plt.imshow(np.array(maze), cmap="binary")
    if path:
        xs = [c for r, c in path]
        ys = [r for r, c in path]
        plt.plot(xs, ys, color="red", linewidth=2, marker="o")
    plt.title(title)
    plt.show()

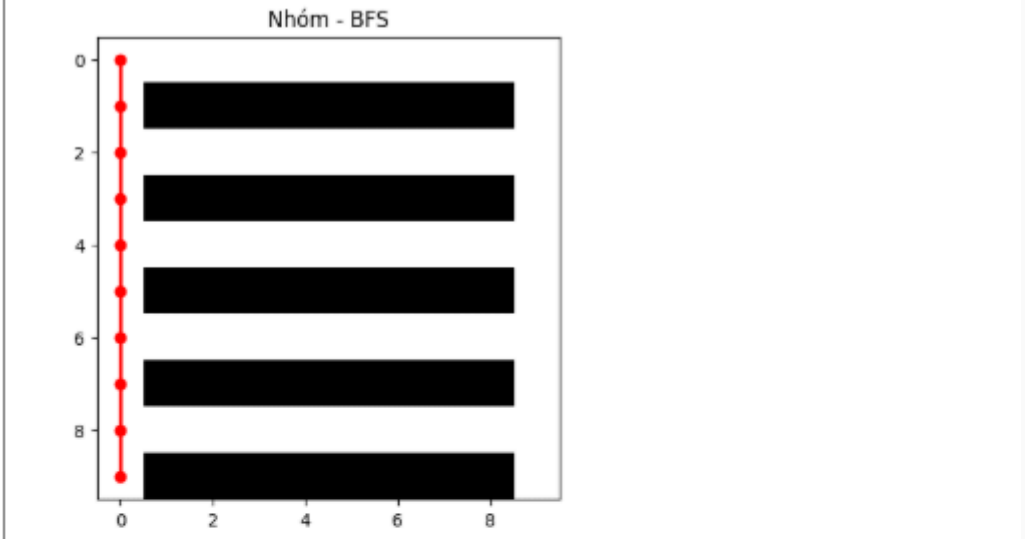
maze_nhom = [
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1, 1, 1, 1, 1, 0],
]
start, goal = (0, 0), (9, 0)

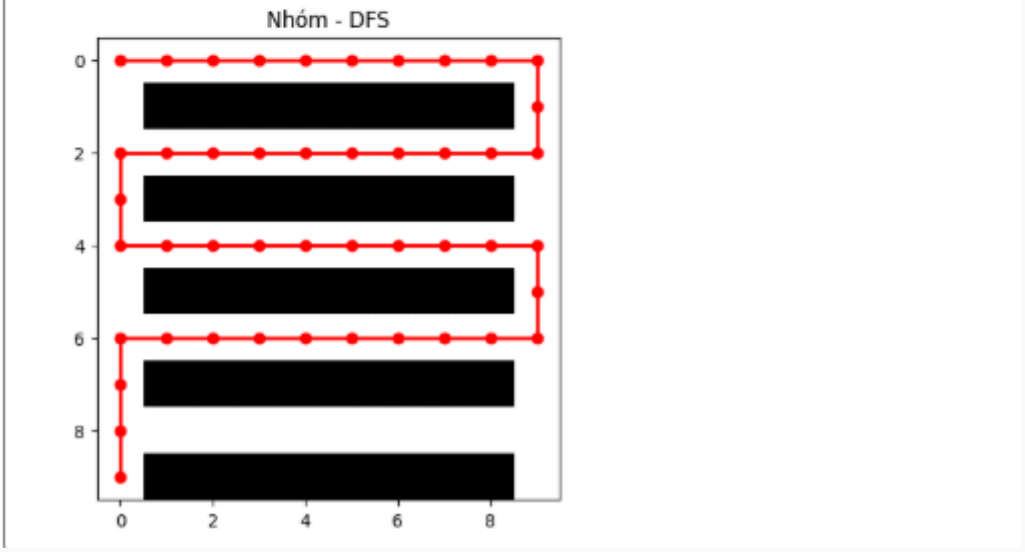
t0 = time.time()
path_bfs, so_node_bfs = bfs(maze_nhom, start, goal)
thoi_gian_bfs = time.time() - t0
t0 = time.time()
path_dfs, so_node_dfs = dfs(maze_nhom, start, goal)
thoi_gian_dfs = time.time() - t0

print("BFS - Số ô duyệt:", so_node_bfs,
      "| Thời gian:", thoi_gian_bfs,
      "| Độ dài đường đi:", len(path_bfs),
      "| Số bước:", len(path_bfs) - 1)
print("DFS - Số ô duyệt:", so_node_dfs,
      "| Thời gian:", thoi_gian_dfs,
      "| Độ dài đường đi:", len(path_dfs),
      "| Số bước:", len(path_dfs) - 1)
ve_mien(maze_nhom, path_bfs, "Nhóm - BFS")
ve_mien(maze_nhom, path_dfs, "Nhóm - DFS")
```

🌾 Thu hoạch: chụp màn hình mê cung của nhóm và kết quả so sánh 2 thuật toán:

```
BFS - Số ô duyệt: 35 | Thời gian: 0.0002353191375732422 | Độ dài đường đi: 10 | Số bước: 9
DFS - Số ô duyệt: 60 | Thời gian: 0.0002725124359130859 4 | Độ dài đường đi: 46 | Số bước: 45
```





Nhận xét của nhóm: trên mê cung do nhóm tự thiết kế, thuật toán nào hiệu quả hơn (ít ô duyệt hơn/nhanh hơn)? Vì sao?

Trên mê cung nhóm tự vẽ, BFS hiệu quả hơn. Nó duyệt 35 ô và đi 9 bước, còn DFS duyệt 60 ô và đi 45 bước. Thời gian trên ảnh gần bằng nhau, khoảng 0,00024 giây và 0,00027 giây, nên chưa kết luận được thuật toán nào nhanh hơn. BFS ít ô và ra đường ngắn vì mở theo khoảng cách: cột 0 đã thông xuống goal `(9,0)`. DFS kém hơn vì ô bên phải được lấy ra trước, nên nó đi hết các hành lang ngang trước khi xuống đích.

TIẾT 5 — ÁP DỤNG MỞ RỘNG: ÁP DỤNG TRÊN CÁC PHƯƠNG PHÁP SAU

Nhóm dựa trên BFS (Breadth-First Search)

Đặc điểm chung: lan tỏa theo từng lớp, đảm bảo tối ưu khi trọng số đồng nhất hoặc không âm (nhờ mở rộng có kiểm soát).

Thuật toán	Đặc điểm	Trọng số	Độ phức tạp
BFS cơ bản	Hàng đợi FIFO, duyệt theo level	Không trọng số / đồng nhất	O(V + E)

0-1 BFS	Deque, cạnh 0 đẩy đầu, cạnh 1 đẩy cuối	0 hoặc 1	O(V + E)
Dial's Algorithm	Dùng mảng bucket thay heap	Số nguyên nhỏ (≤ C)	O(E + V·C)
Dijkstra	Mở rộng BFS bằng priority queue — luôn chọn đỉnh có dist nhỏ nhất	Không âm	O((V+E) log V)
A*	Dijkstra + heuristic h(n) để ưu tiên đỉnh gần đích	Không âm	Phụ thuộc heuristic
Multi-source BFS	Nhiều nguồn cùng lúc, lan tỏa đồng thời	Không trọng số	O(V + E)
Bidirectional BFS	BFS từ cả nguồn và đích, gặp nhau ở giữa	Không trọng số	O(b^(d/2))

Nhận xét: Dijkstra và A* thực chất là "BFS có trọng số" — vẫn là lan tỏa theo lớp, nhưng lớp được xác định bởi dist thay vì số cạnh.

Nhóm dựa trên DFS (Depth-First Search)

Đặc điểm chung: đi sâu rồi quay lui. Không đảm bảo đường đi ngắn nhất trên đồ thị tổng quát, nhưng dùng được cho các bài toán cấu trúc hoặc đồ thị đặc biệt.

Thuật toán / Kỹ thuật	Ứng dụng	Ghi chú
DFS cơ bản	Kiểm tra có đường đi, tìm 1 đường bất kỳ	Không tối ưu
DFS trên cây	Đường đi ngắn nhất trên cây	Đúng vì cây chỉ có 1 đường
DFS + memoization	Đường đi ngắn nhất trên DAG	Quy hoạch động trên DAG
Topological sort + DP	Đường đi ngắn nhất trên DAG có trọng số (kể cả âm)	O(V + E)
IDA* (Iterative Deepening A*)	DFS có giới hạn + heuristic	Bộ nhớ thấp hơn A*
DFS phát hiện chu trình âm	Hỗ trợ Bellman–Ford / Johnson	Tiền xử lý

Tarjan / Kosaraju (DFS-based)	Tìm SCC, cầu, khớp	Không trực tiếp tìm đường ngắn nhất

Nhận xét: DFS không phải công cụ chính để tìm đường ngắn nhất, nhưng DFS trên DAG (kết hợp topo sort + DP) là một phương pháp rất hiệu quả và xử lý được cả trọng số âm.

Nhóm lai / mở rộng (Hybrid)

Không thuần BFS hay DFS, mà kết hợp nhiều chiến lược hoặc dùng cả hai.

Thuật toán	Chiến lược	Ghi chú
Bellman–Ford	Duyệt lặp toàn bộ cạnh V−1 lần	Không thuần BFS/DFS; phát hiện chu trình âm
SPFA	Bellman–Ford + queue (giống BFS)	Lai giữa Bellman–Ford và BFS
Floyd–Warshall	Quy hoạch động 3 vòng lặp	Không duyệt đồ thị theo BFS/DFS
Johnson	Bellman–Ford (reweight) + Dijkstra	Kết hợp 2 phương pháp
Bidirectional Dijkstra	Dijkstra từ 2 phía	Mở rộng của BFS có trọng số

5 BÀI THỰC HÀNH THÊM DÀNH CHO SINH VIÊN CNTT

Năm bài dưới đây áp dụng tìm kiếm mù (BFS, DFS) vào các bài toán quen thuộc của ngành CNTT: định tuyến mạng, web crawler, phụ thuộc gói phần mềm, đo hiệu năng thuật toán và lịch sử Git. Mỗi bài có code chạy sẵn, câu hỏi Suy nghĩ và yêu cầu Cải tiến.

Bài 1 — BFS tìm đường đi ngắn nhất trong mạng nội bộ và phát hiện điểm đơn lỗi

Liên hệ CNTT: lệnh traceroute và các giao thức định tuyến đều cần tìm đường đi ít bước nhảy nhất giữa các thiết bị; BFS chính là thuật toán cho bài toán đồ thị không trọng số này. Bài này còn mô phỏng sự cố đứt liên kết để thấy BFS tự tìm đường vòng hoặc báo thiết bị mất kết nối. (Sơ đồ mạng giả lập.)

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
from collections import deque

# ---- Sơ đồ mạng nội bộ: mỗi dòng là thiết bị và các thiết bị nối trực tiếp với nó
mang = {
    "GW":     ["SW1", "SW2"],
    "SW1":    ["GW", "SRV1", "SRV2", "SW3"],
    "SW2":    ["GW", "SW3", "DB"],
    "SW3":    ["SW1", "SW2", "PC1", "PC2"],
    "SRV1":   ["SW1"],
    "SRV2":   ["SW1", "DB"],
    "DB":     ["SW2", "SRV2", "BACKUP"],
    "BACKUP": ["DB"],
    "PC1":    ["SW3"],
    "PC2":    ["SW3", "PRINTER"],
    "PRINTER": ["PC2"],
```

```
    "CAM":   [],          # camera chưa được nối vào mạng
}

def bfs_duong_di(do_thi, bat_dau, dich):
    """Trả về đường đi ít bước nhảy (hop) nhất, hoặc None nếu không đến được"""
    hang_doi = deque([[bat_dau]])
    da_tham = {bat_dau}
    while hang_doi:
        duong = hang_doi.popleft()
        nut = duong[-1]
        if nut == dich:
            return duong
        for ke in do_thi[nut]:
            if ke not in da_tham:
                da_tham.add(ke)
                hang_doi.append(duong + [ke])
    return None

def tap_den_duoc(do_thi, bat_dau):
    da_tham = {bat_dau}
    hang_doi = deque([bat_dau])
    while hang_doi:
        nut = hang_doi.popleft()
        for ke in do_thi[nut]:
            if ke not in da_tham:
                da_tham.add(ke)
                hang_doi.append(ke)
    return da_tham

duong = bfs_duong_di(mang, "GW", "BACKUP")
print("Đường ngắn nhất từ GW đến BACKUP:", " -> ".join(duong), f"({len(duong) - 1} bước
nhảy)")

# ---- Sự cố: đứt kết nối giữa SW2 và DB
def cat_lien_ket(do_thi, a, b):
    moi = {k: list(v) for k, v in do_thi.items()}
    moi[a].remove(b)
    moi[b].remove(a)
    return moi

mang_su_co = cat_lien_ket(mang, "SW2", "DB")
duong2 = bfs_duong_di(mang_su_co, "GW", "BACKUP")
print("Sau khi đứt SW2-DB:", " -> ".join(duong2), f"({len(duong2) - 1} bước nhảy)")

# ---- Sự cố nặng hơn: đứt thêm SRV2-DB
mang_su_co2 = cat_lien_ket(mang_su_co, "SRV2", "DB")
print("Sau khi đứt thêm SRV2-DB:", bfs_duong_di(mang_su_co2, "GW", "BACKUP"))
mat_lien_lac = sorted(set(mang) - tap_den_duoc(mang_su_co2, "GW"))
print("Thiết bị không còn truy cập được từ GW:", mat_lien_lac)
```

```
    while hang_doi:
        duong = hang_doi.popleft()
        nut = duong[-1]
        if nut == dich:
            return duong
        for ke in do_thi[nut]:
            if ke not in da_tham:
                da_tham.add(ke)
                hang_doi.append(duong + [ke])
    return None

def tap_den_duoc(do_thi, bat_dau):
    da_tham = {bat_dau}
    hang_doi = deque([bat_dau])
    while hang_doi:
        nut = hang_doi.popleft()
        for ke in do_thi[nut]:
            if ke not in da_tham:
                da_tham.add(ke)
                hang_doi.append(ke)
    return da_tham

duong = bfs_duong_di(mang, "GW", "BACKUP")
print("Đường ngắn nhất từ GW đến BACKUP:", " -> ".join(duong), f"({len(duong) - 1} bước nhảy)")

# ---- Sự cố: đứt kết nối giữa SW2 và DB
def cat_lien_ket(do_thi, a, b):
    moi = {k: list(v) for k, v in do_thi.items()}
    moi[a].remove(b)
    moi[b].remove(a)
    return moi

mang_su_co = cat_lien_ket(mang, "SW2", "DB")
duong2 = bfs_duong_di(mang_su_co, "GW", "BACKUP")
print("Sau khi đứt SW2-DB:", " -> ".join(duong2), f"({len(duong2) - 1} bước nhảy)")

# ---- Sự cố nặng hơn: đứt thêm SRV2-DB
mang_su_co2 = cat_lien_ket(mang_su_co, "SRV2", "DB")
print("Sau khi đứt thêm SRV2-DB:", bfs_duong_di(mang_su_co2, "GW", "BACKUP"))
mat_lien_lac = sorted(set(mang) - tap_den_duoc(mang_su_co2, "GW"))
print("Thiết bị không còn truy cập được từ GW:", mat_lien_lac)
```

```
Đường ngắn nhất từ GW đến BACKUP: GW -> SW2 -> DB -> BACKUP (3 bước nhảy)
Sau khi đứt SW2-DB: GW -> SW1 -> SRV2 -> DB -> BACKUP (4 bước nhảy)
Sau khi đứt thêm SRV2-DB: None
Thiết bị không còn truy cập được từ GW: ['BACKUP', 'CAM', 'DB']
```

💭 **Suy nghĩ: Vì sao BFS luôn tìm được đường có ít bước nhảy nhất? Sau khi đứt liên kết SW2–DB, đường đi thay đổi thế nào và vì sao dài hơn? Khi đứt thêm SRV2–DB thì những thiết bị nào mất kết nối, và vì sao camera (CAM) luôn nằm trong danh sách?**

BFS luôn ra đường ít bước nhảy nhất vì mọi đường dài k bước được xét xong trước đường dài k+1, và `visited` không cho đi lại nút đã có đường ngắn hơn.

Trước sự cố, đường là `GW → SW2 → DB → BACKUP`, 3 bước nhảy. Đứt SW2–DB thì mất đường thẳng đó. BFS tự vòng `GW → SW1 → SRV2 → DB → BACKUP`, thành 4 bước, vì phải đi qua SRV2 mới tới DB.

Đứt thêm SRV2–DB thì không còn đường nào tới DB, nên `bfs_duong_di` trả về `None`. Mất kết nối từ GW: `BACKUP`, `CAM`, `DB`. CAM luôn nằm trong danh sách vì trong sơ đồ `CAM = []` — camera chưa được nối mạng từ đầu, không phải do hai sự cố này.

🔧 Cải tiến: Viết một vòng lặp thử đứt LẦN LƯỢT từng liên kết của mạng và in ra những liên kết mà nếu đứt thì GW không còn đến được BACKUP (điểm đơn lỗi — single point of failure). Sau đó thêm một liên kết dự phòng SW1–DB vào sơ đồ, chạy lại và nhận xét đường đi và các điểm đơn lỗi thay đổi thế nào.

```
from collections import deque


mang = {
    "GW": ["SW1", "SW2"],
```

```
    "SW1": ["GW", "SRV1", "SRV2", "SW3"],
    "SW2": ["GW", "SW3", "DB"],
    "SW3": ["SW1", "SW2", "PC1", "PC2"],
    "SRV1": ["SW1"],
    "SRV2": ["SW1", "DB"],
    "DB": ["SW2", "SRV2", "BACKUP"],
    "BACKUP": ["DB"],
    "PC1": ["SW3"],
    "PC2": ["SW3", "PRINTER"],
    "PRINTER": ["PC2"],
    "CAM": [],
}


def bfs_duong_di(do_thi, bat_dau, dich):
    hang_doi = deque([[bat_dau]])
    da_tham = {bat_dau}
    while hang_doi:
        duong = hang_doi.popleft()
        nut = duong[-1]
        if nut == dich:
            return duong
        for ke in do_thi[nut]:
            if ke not in da_tham:
                da_tham.add(ke)
                hang_doi.append(duong + [ke])
    return None


def cat_lien_ket(do_thi, a, b):
    moi = {k: list(v) for k, v in do_thi.items()}
    if b in moi[a]:
```

```
        moi[a].remove(b)
    if a in moi[b]:
        moi[b].remove(a)
    return moi


def cac_canh(do_thi):
    canh = set()
    for a, ks in do_thi.items():
        for b in ks:
            canh.add(tuple(sorted((a, b))))
    return canh


def diem_don_loi(do_thi, bat_dau="GW", dich="BACKUP"):
    ket_qua = []
    for a, b in sorted(cac_canh(do_thi)):
        if bfs_duong_di(cat_lien_ket(do_thi, a, b), bat_dau, dich) is None:
            ket_qua.append((a, b))
    return ket_qua


print("Đường gốc:", " -> ".join(bfs_duong_di(mang, "GW", "BACKUP")))
print("Điểm đơn lỗi ban đầu:")
for a, b in diem_don_loi(mang):
    print(f"  {a} -- {b}")


mang_dp = {k: list(v) for k, v in mang.items()}
mang_dp["SW1"].append("DB")
mang_dp["DB"].append("SW1")
duong_moi = bfs_duong_di(mang_dp, "GW", "BACKUP")
print("Đường sau khi thêm SW1-DB:", " -> ".join(duong_moi),
      f"({len(duong_moi) - 1} bước nhảy)")
```

print("Điểm đơn lỗi sau khi thêm dự phòng:")

for a, b in diem_don_loi(mang_dp):

print(f" {a} -- {b}")

```
                ket_qua.append((a, b))
        return ket_qua

    print("Đường gốc:", " -> ".join(bfs_duong_di(mang, "GW", "BACKUP")))
    print("Điểm đơn lỗi ban đầu:")
    for a, b in diem_don_loi(mang):
        print(f"  {a} -- {b}")

    mang_dp = {k: list(v) for k, v in mang.items()}
    mang_dp["SW1"].append("DB")
    mang_dp["DB"].append("SW1")
    duong_moi = bfs_duong_di(mang_dp, "GW", "BACKUP")
    print("Đường sau khi thêm SW1-DB:", " -> ".join(duong_moi),
          f"({len(duong_moi) - 1} bước nhảy)")
    print("Điểm đơn lỗi sau khi thêm dự phòng:")
    for a, b in diem_don_loi(mang_dp):
        print(f"  {a} -- {b}")
```

```
Đường gốc: GW -> SW2 -> DB -> BACKUP
Điểm đơn lỗi ban đầu:
  BACKUP -- DB
Đường sau khi thêm SW1-DB: GW -> SW1 -> DB -> BACKUP (3 bước nhảy)
Điểm đơn lỗi sau khi thêm dự phòng:
  BACKUP -- DB
```

Bài 2 — Web crawler bằng BFS: giới hạn độ sâu và vai trò của tập "đã thăm"

Liên hệ CNTT: trình thu thập dữ liệu web (crawler) duyệt các trang theo từng tầng liên kết từ trang gốc. Hai điều bắt buộc: giới hạn độ sâu (để không thu thập cả internet) và ghi nhớ trang đã thăm (vì các trang liên kết vòng với nhau). (Website giả lập 60 trang.)

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import random
from collections import deque

# ---- Website giả lập: 60 trang, mỗi trang có 1-4 liên kết ra các trang khác (có cả liên kết vòng)
rng = random.Random(4)
SO_TRANG = 60
site = {f"p{i}": [] for i in range(SO_TRANG)}
for i in range(SO_TRANG):
    for _ in range(rng.randint(1, 4)):
        j = rng.randrange(SO_TRANG)
        if j != i and f"p{j}" not in site[f"p{i}"]:
            site[f"p{i}"].append(f"p{j}")

def crawl_bfs(site, goc, do_sau_toi_da):
    """Thu thập theo từng tầng (BFS), có tập 'đã thăm' để không tải một trang hai lần"""
    da_tham = {goc: 0}
    hang_doi = deque([goc])
    while hang_doi:
        trang = hang_doi.popleft()
        if da_tham[trang] == do_sau_toi_da:
            continue
        for lk in site[trang]:
            if lk not in da_tham:
                da_tham[lk] = da_tham[trang] + 1
                hang_doi.append(lk)
    return da_tham

for d in range(0, 6):
    kq = crawl_bfs(site, "p0", d)
```

```
    print(f"Độ sâu tối đa {d}: thu thập {len(kq):>2} trang")

kq = crawl_bfs(site, "p0", 10)
theo_tang = {}
for trang, tang in kq.items():
    theo_tang.setdefault(tang, []).append(trang)
print("\nSố trang theo từng tầng:", {t: len(v) for t, v in sorted(theo_tang.items())})
print(f"Trang không thể đến được từ p0: {SO_TRANG - len(kq)}")

# ---- Điều gì xảy ra nếu KHÔNG ghi nhớ trang đã thăm (đặt giới hạn 20.000 lượt tải để không
treo máy)
so_lan_tai = 0
hang_doi = deque([("p0", 0)])
while hang_doi and so_lan_tai < 20000:
    trang, tang = hang_doi.popleft()
    so_lan_tai += 1
    if tang < 8:
        for lk in site[trang]:
            hang_doi.append((lk, tang + 1))
print(f"\nKhông có tập 'đã thăm', độ sâu 8: tải {so_lan_tai} lượt (trong khi chỉ có
{len(crawl_bfs(site, 'p0', 8))} trang khác nhau)")
```

```
        for lk in site[trang]:
            hang_doi.append((lk, tang + 1))
print(f"\nKhông có tập 'đã thăm', độ sâu 8: tải {so_lan_tai} lượt (trong khi chỉ có {len(crawl_bfs(site, 'p0', 8))} trang khác nhau)")
```

```
Độ sâu tối đa 0: thu thập  1 trang
Độ sâu tối đa 1: thu thập  3 trang
Độ sâu tối đa 2: thu thập  5 trang
Độ sâu tối đa 3: thu thập 10 trang
Độ sâu tối đa 4: thu thập 22 trang
Độ sâu tối đa 5: thu thập 35 trang

Số trang theo từng tầng: {0: 1, 1: 2, 2: 2, 3: 5, 4: 12, 5: 13, 6: 11, 7: 8, 8: 3}
Trang không thể đến được từ p0: 3

Không có tập 'đã thăm', độ sâu 8: tải 641 lượt (trong khi chỉ có 57 trang khác nhau)
```

💭 **Suy nghĩ: Số trang thu thập được tăng thế nào khi độ sâu tăng từ 0 đến 5, và vì sao tăng nhanh ở các tầng giữa rồi chậm lại? Nếu không có tập "đã thăm" thì số lượt tải thay đổi ra sao, và điều này gây hại gì cho máy chủ bị thu thập? Vì sao BFS phù hợp hơn DFS để thu thập "các trang quan trọng gần trang gốc" trước?**

Từ độ sâu 0 đến 5, số trang thu được là 1, 3, 5, 10, 22, 35. Lúc đầu tăng chậm vì `p0` chỉ có vài liên kết. Tầng giữa tăng nhanh vì mỗi trang mới lại mở thêm nhiều trang chưa thăm. Sau đó chậm lại vì phần lớn trang đã nằm trong tập đã thăm, và có 3 trang không có đường từ `p0`.

Không có tập đã thăm, độ sâu 8 tải 641 lượt trong khi chỉ có 57 trang khác nhau. Liên kết vòng làm cùng một trang bị xếp hàng nhiều lần. Máy chủ phải trả lại nội dung cũ, tốn băng thông, và có thể bị coi là tải bất thường.

BFS phù hợp hơn vì trang gần gốc nằm ở tầng nhỏ và được duyệt trước. DFS có thể cắm sâu một chuỗi link trước khi quay lại các trang kề trang gốc.

🔧 Cải tiến: Đổi hạt giống random.Random(4) thành 5 rồi 6 và nhận xét số trang theo tầng. Sau đó thêm tham số so_trang_toi_da vào crawl_bfs (dừng khi đã thu thập đủ, ví dụ 30 trang) và thử cài thêm bộ lọc chỉ nhận liên kết có số trang chẵn để mô phỏng việc chỉ thu thập một tên miền.

```
import random

from collections import deque



SO_TRANG = 60



def tao_site(seed):

    rng = random.Random(seed)
```

```
    site = {f"p{i}": [] for i in range(SO_TRANG)}
    for i in range(SO_TRANG):
        for _ in range(rng.randint(1, 4)):
            j = rng.randrange(SO_TRANG)
            if j != i and f"p{j}" not in site[f"p{i}"]:
                site[f"p{i}"].append(f"p{j}")
    return site


def crawl_bfs(site, goc, do_sau_toi_da, so_trang_toi_da=None, loc=None):
    da_tham = {goc: 0}
    hang_doi = deque([goc])
    while hang_doi:
        if so_trang_toi_da is not None and len(da_tham) >= so_trang_toi_da:
            break
        trang = hang_doi.popleft()
        if da_tham[trang] == do_sau_toi_da:
            continue
        for lk in site[trang]:
            if loc is not None and not loc(lk):
                continue
            if lk not in da_tham:
                da_tham[lk] = da_tham[trang] + 1
                hang_doi.append(lk)
                if so_trang_toi_da is not None and len(da_tham) >= so_trang_toi_da:
                    break
    return da_tham


def in_tang(seed):
    kq = crawl_bfs(tao_site(seed), "p0", 10)
    theo_tang = {}
```

```
    for trang, tang in kq.items():
        theo_tang[tang] = theo_tang.get(tang, 0) + 1
    print(f"seed={seed} | tổng {len(kq)} | theo tầng: {dict(sorted(theo_tang.items()))}")

for seed in (5, 6):
    in_tang(seed)

site = tao_site(4)
kq30 = crawl_bfs(site, "p0", 10, so_trang_toi_da=30)
print("Giới hạn 30 trang:", len(kq30), "| sâu lớn nhất:", max(kq30.values()))

def loc_trang_chan(lk):
    return int(lk[1:]) % 2 == 0

kq_chan = crawl_bfs(site, "p0", 10, loc=loc_trang_chan)
print("Chỉ lấy trang chẵn:", len(kq_chan), "|", sorted(kq_chan))
```

```
print( Chỉ lấy trang chẵn: , len(kq_chan),  | , sorted(kq_chan))
```

```
seed=5 | tổng 54 | theo tầng: {0: 1, 1: 3, 2: 9, 3: 17, 4: 16, 5: 5, 6: 3}
seed=6 | tổng 58 | theo tầng: {0: 1, 1: 1, 2: 4, 3: 5, 4: 6, 5: 12, 6: 15, 7: 8, 8: 2, 9: 3, 10: 1}
Giới hạn 30 trang: 30 | sâu lớn nhất: 5
Chỉ lấy trang chẵn: 7 | ['p0', 'p12', 'p16', 'p26', 'p36', 'p40', 'p6']
```

Bài 3 — DFS giải bài toán phụ thuộc gói phần mềm: thứ tự cài đặt và phát hiện vòng

Liên hệ CNTT: pip, npm, apt đều phải xác định thứ tự cài đặt các gói sao cho gói con được cài trước gói cha, và báo lỗi khi có vòng phụ thuộc. Đây là ứng dụng kinh điển của DFS (sắp xếp tô-pô, topological sort). (Cây phụ thuộc giả lập.)

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
# ---- Trình quản lý gói: mỗi gói liệt kê các gói nó PHỤ THUỘC vào
phu_thuoc = {
    "webapp":      ["flask", "sqlalchemy", "celery"],
    "flask":       ["werkzeug", "jinja2"],
    "jinja2":      ["markupsafe"],
    "werkzeug":    ["markupsafe"],
    "sqlalchemy":  ["greenlet"],
    "celery":      ["kombu", "sqlalchemy"],
    "kombu":       ["amqp"],
    "amqp":        [],
    "markupsafe":  [],
    "greenlet":    [],
}

def thu_tu_cai_dat(phu_thuoc, muc_tieu):
```

```
    """DFS: cài các gói con trước, gói cha sau. Phát hiện vòng phụ thuộc bằng trạng thái 3
màu."""
    CHUA, DANG_XET, XONG = 0, 1, 2
    trang_thai = {g: CHUA for g in phu_thuoc}
    ket_qua, ngan_xep = [], []

    def tham(g):
        if trang_thai[g] == XONG:
            return
        if trang_thai[g] == DANG_XET:
            vong = ngan_xep[ngan_xep.index(g):] + [g]
            raise ValueError("Phát hiện vòng phụ thuộc: " + " -> ".join(vong))
        trang_thai[g] = DANG_XET
        ngan_xep.append(g)
        for con in phu_thuoc[g]:
            tham(con)
        ngan_xep.pop()
        trang_thai[g] = XONG
        ket_qua.append(g)

    tham(muc_tieu)
    return ket_qua

thu_tu = thu_tu_cai_dat(phu_thuoc, "webapp")
print("Thứ tự cài đặt hợp lệ:")
for i, g in enumerate(thu_tu, 1):
    print(f"  {i:>2}. {g}")

# ---- Kiểm tra: mọi gói phải được cài SAU các gói nó phụ thuộc
vi_tri = {g: i for i, g in enumerate(thu_tu)}
assert all(vi_tri[con] < vi_tri[g] for g in thu_tu for con in phu_thuoc[g]), "Thứ tự sai!"
print("\nKiểm tra thứ tự: OK")

# ---- Gây ra vòng phụ thuộc: markupsafe lại phụ thuộc vào flask
phu_thuoc_loi = {k: list(v) for k, v in phu_thuoc.items()}
phu_thuoc_loi["markupsafe"].append("flask")
try:
    thu_tu_cai_dat(phu_thuoc_loi, "webapp")
except ValueError as loi:
    print("\n" + str(loi))
```

```
thu_tu = thu_tu_cai_dat(phu_thuoc, "webapp")
print("Thứ tự cài đặt hợp lệ:")
for i, g in enumerate(thu_tu, 1):
    print(f"  {i:>2}. {g}")

# ---- Kiểm tra: mọi gói phải được cài SAU các gói nó phụ thuộc
vi_tri = {g: i for i, g in enumerate(thu_tu)}
assert all(vi_tri[con] < vi_tri[g] for g in thu_tu for con in phu_thuoc[g]), "Thứ tự sai!"
print("\nKiểm tra thứ tự: OK")

# ---- Gây ra vòng phụ thuộc: markupsafe lại phụ thuộc vào flask
phu_thuoc_loi = {k: list(v) for k, v in phu_thuoc.items()}
phu_thuoc_loi["markupsafe"].append("flask")
try:
    thu_tu_cai_dat(phu_thuoc_loi, "webapp")
except ValueError as loi:
    print("\n" + str(loi))
```

```
Thứ tự cài đặt hợp lệ:
   1. markupsafe
   2. werkzeug
   3. jinja2
   4. flask
   5. greenlet
   6. sqlalchemy
   7. amqp
   8. kombu
   9. celery
  10. webapp

Kiểm tra thứ tự: OK

Phát hiện vòng phụ thuộc: flask -> werkzeug -> markupsafe -> flask
```

💭 **Suy nghĩ: Vì sao DFS thêm một gói vào kết quả SAU khi đã xử lý hết các gói con của nó? Ba trạng thái (chưa xét, đang xét, xong) giúp phát hiện vòng thế nào? Thứ tự cài đặt hợp lệ có duy nhất không, và điều đó có ý nghĩa gì khi cài song song nhiều gói?**

DFS thêm một gói vào kết quả sau khi đã xử lý hết gói con, vì đây là post-order: gói con phải được cài trước thì gói cha mới cài được. `pip` và `npm` cũng cần thứ tự đó.

Ba trạng thái dùng để phân biệt việc đang duyệt và việc đã xong. Chưa xét thì đi vào. Đang xét mà gặp lại nghĩa là gói vẫn nằm trên ngăn xếp, tức có vòng, như `flask -> werkzeug -> markupsafe -> flask`. Xong thì bỏ qua, không duyệt lại.

Thứ tự hợp lệ không duy nhất. `werkzeug` và `jinja2` đều chỉ cần `markupsafe`, đổi thứ tự con của `flask` là hai gói này đổi chỗ. Khi cài song song, các gói không phụ thuộc nhau có thể cài cùng lúc, miễn là vẫn đứng sau mọi gói con của chúng.

🔧 Cải tiến: Thêm hàm goi_bi_anh_huong(phu_thuoc, goi_loi) trả về tất cả gói bị ảnh hưởng nếu một gói bị lỗi (phụ thuộc ngược, dùng BFS hoặc DFS trên đồ thị đảo chiều). Thử với markupsafe và greenlet. Sau đó thêm một gói mới pytest phụ thuộc vào pluggy và flask rồi cài lại.

```
from collections import deque


phu_thuoc = {
    "webapp": ["flask", "sqlalchemy", "celery"],
    "flask": ["werkzeug", "jinja2"],
    "jinja2": ["markupsafe"],
    "werkzeug": ["markupsafe"],
    "sqlalchemy": ["greenlet"],
    "celery": ["kombu", "sqlalchemy"],
    "kombu": ["amqp"],
    "amqp": [],
    "markupsafe": [],
    "greenlet": [],
}


def thu_tu_cai_dat(phu_thuoc, muc_tieu):
    CHUA, DANG_XET, XONG = 0, 1, 2
    trang_thai = {g: CHUA for g in phu_thuoc}
    ket_qua, ngan_xep = [], []


    def tham(g):
```

```
        if trang_thai[g] == XONG:
            return
        if trang_thai[g] == DANG_XET:
            vong = ngan_xep[ngan_xep.index(g):] + [g]
            raise ValueError("Phát hiện vòng phụ thuộc: " + " -> ".join(vong))
        trang_thai[g] = DANG_XET
        ngan_xep.append(g)
        for con in phu_thuoc[g]:
            tham(con)
        ngan_xep.pop()
        trang_thai[g] = XONG
        ket_qua.append(g)

    tham(muc_tieu)
    return ket_qua


def goi_bi_anh_huong(phu_thuoc, goi_loi):
    """BFS trên đồ thị ngược: gói nào phụ thuộc, trực tiếp hoặc gián tiếp, vào goi_loi."""
    dao = {g: [] for g in phu_thuoc}
    for g, cons in phu_thuoc.items():
        for c in cons:
            dao.setdefault(c, []).append(g)
    hang_doi = deque([goi_loi])
    thay = {goi_loi}
    while hang_doi:
        u = hang_doi.popleft()
        for v in dao.get(u, []):
            if v not in thay:
                thay.add(v)
                hang_doi.append(v)
```

```
    thay.discard(goi_loi)

    return sorted(thay)


print("markupsafe lỗi, ảnh hưởng:", goi_bi_anh_huong(phu_thuoc, "markupsafe"))

print("greenlet lỗi, ảnh hưởng:", goi_bi_anh_huong(phu_thuoc, "greenlet"))


phu_thuoc["pytest"] = ["pluggy", "flask"]

phu_thuoc["pluggy"] = []

thu_tu = thu_tu_cai_dat(phu_thuoc, "pytest")

print("Thứ tự cài pytest:")

for i, g in enumerate(thu_tu, 1):

    print(f"  {i:>2}. {g}")
```

```
    thay.discard(goi_loi)
    return sorted(thay)

print("markupsafe lỗi, ảnh hưởng:", goi_bi_anh_huong(phu_thuoc, "markupsafe"))
print("greenlet lỗi, ảnh hưởng:", goi_bi_anh_huong(phu_thuoc, "greenlet"))

phu_thuoc["pytest"] = ["pluggy", "flask"]
phu_thuoc["pluggy"] = []
thu_tu = thu_tu_cai_dat(phu_thuoc, "pytest")
print("Thứ tự cài pytest:")
for i, g in enumerate(thu_tu, 1):
    print(f"  {i:>2}. {g}")
```

```
markupsafe lỗi, ảnh hưởng: ['flask', 'jinja2', 'webapp', 'werkzeug']
greenlet lỗi, ảnh hưởng: ['celery', 'sqlalchemy', 'webapp']
Thứ tự cài pytest:
   1. pluggy
   2. markupsafe
   3. werkzeug
   4. jinja2
   5. flask
   6. pytest
```

Bài 4 — Đo chi phí BFS và DFS khi kích thước bài toán tăng: số ô khám phá, bộ nhớ, thời gian

Liên hệ CNTT: sinh viên CNTT cần đánh giá thuật toán bằng thực nghiệm chứ không chỉ bằng lý thuyết. Bài này đo trên lưới ngẫu nhiên từ 10x10 đến 80x80: số ô đã khám phá, độ dài đường tìm được, kích thước lớn nhất của hàng chờ (đại diện cho bộ nhớ) và thời gian chạy. (Lưới sinh bằng hạt giống cố định nên kết quả lặp lại được.)

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
import random, time
from collections import deque
```

```
def tao_luoi(n, ty_le_tuong, seed):
    """Lưới n x n ngẫu nhiên; thử các seed liên tiếp cho đến khi từ góc trên-trái đến góc
dưới-phải có đường đi"""
    while True:
        rng = random.Random(seed)
        luoi = [[1 if rng.random() < ty_le_tuong else 0 for _ in range(n)] for _ in range(n)]
        luoi[0][0] = luoi[n - 1][n - 1] = 0
        if tim(luoi, "bfs")[0] is not None:
            return luoi
        seed += 1

def tim(luoi, kieu):
    n = len(luoi)
    dich = (n - 1, n - 1)
    cho = deque([(0, 0)]) if kieu == "bfs" else [(0, 0)]
    cha = {(0, 0): None}
    dinh_cho = 1
    while cho:
        r, c = cho.popleft() if kieu == "bfs" else cho.pop()
        if (r, c) == dich:
            break
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nr, nc = r + dr, c + dc
            if 0 <= nr < n and 0 <= nc < n and luoi[nr][nc] == 0 and (nr, nc) not in cha:
                cha[(nr, nc)] = (r, c)
                cho.append((nr, nc))
        dinh_cho = max(dinh_cho, len(cho))
    if dich not in cha:
        return None, len(cha), dinh_cho
    duong, nut = [], dich
    while nut is not None:
        duong.append(nut)
        nut = cha[nut]
    return duong, len(cha), dinh_cho

print(f"{'kích thước':>10}{'thuật toán':>12}{'ô đã khám phá':>15}{'độ dài đường':>14}{'hàng
chờ lớn nhất':>19}{'thời gian (ms)':>16}")
for n in [10, 20, 40, 80]:
    luoi = tao_luoi(n, 0.22, seed=n)
    for kieu in ["bfs", "dfs"]:
        t = time.perf_counter()
        duong, so_o, dinh = tim(luoi, kieu)
        ms = (time.perf_counter() - t) * 1000
        print(f"{n:>5}x{n:<4}{kieu.upper():>12}{so_o:>15}{len(duong) -
1:>14}{dinh:>19}{ms:>16.2f}")
```


```
        while nut is not None:
            duong.append(nut)
            nut = cha[nut]
    return duong, len(cha), dinh_cho

print(f"{'kích thước':>10}{'thuật toán':>12}{'ô đã khám phá':>15}{'độ dài đường':>14}{'hàng chờ lớn nhất':>19}{'thời gian (ms)':>16}")
for n in [10, 20, 40, 80]:
    luoi = tao_luoi(n, 0.22, seed=n)
    for kieu in ["bfs", "dfs"]:
        t = time.perf_counter()
        duong, so_o, dinh = tim(luoi, kieu)
        ms = (time.perf_counter() - t) * 1000
        print(f"{n:>5}x{n:<4}{kieu.upper():>12}{so_o:>15}{len(duong) - 1:>14}{dinh:>19}{ms:>16.2f}")
```


```
    kích thước  thuật toán  ô đã khám phá  độ dài đường  hàng chờ lớn nhất  thời gian (ms)
       10x10          BFS             79            18                 11           0.11
       10x10          DFS             75            22                 16           0.09
       20x20          BFS            224            38                 15           0.29
       20x20          DFS            140            52                 44           0.15
       40x40          BFS           1220            78                 42           1.66
       40x40          DFS            583           206                184           0.58
       80x80          BFS           5003           158                 82           8.34
       80x80          DFS           3051           508                681           4.98
```

💭 **Suy nghĩ: Đường đi BFS và DFS khác nhau thế nào về độ dài khi lưới lớn dần, và vì sao BFS luôn tối ưu còn DFS thì không? Hàng chờ lớn nhất của thuật toán nào lớn hơn ở lưới 80x80, và điều này trái với điều thường nghe ("DFS tốn ít bộ nhớ") ở điểm nào? Vì sao DFS khám phá ít ô hơn nhưng vẫn không nên chọn khi cần đường ngắn nhất?**

Lưới càng lớn, hai đường càng tách. 10×10: BFS 18 bước, DFS 22. 40×40: 78 và 206. 80×80: 158 và 508. BFS luôn tối ưu vì mỗi cạnh giá 1, lần đầu tới đích là ngắn nhất. DFS dừng ở nhánh chạm đích trước, nên không tối ưu.

Ở 80×80, hàng chờ lớn nhất của DFS là 681, của BFS chỉ 82. Câu "DFS tốn ít bộ nhớ" đúng trên cây có nhánh rẽ lớn: stack theo độ sâu, queue theo cả một tầng. Trên lưới, BFS chỉ giữ vành đai của sóng, còn DFS đẩy nhiều ô kề vào stack trước khi đi sâu, nên frontier có thể lớn hơn.

DFS khám phá ít ô hơn, 3051 so với 5003, vì dừng sớm. Vẫn không chọn DFS khi cần đường ngắn: ít ô duyệt không bù được đường dài gần gấp ba.

🔧 Cải tiến: Đổi ty_le_tuong từ 0.22 thành 0.1 rồi 0.3 và nhận xét ảnh hưởng đến số ô khám phá, độ dài đường và hàng chờ của hai thuật toán. Sau đó thêm kích thước 120 vào danh sách và quan sát thời gian chạy tăng theo n như thế nào (gợi ý: so sánh với số ô n×n).

```
import random
import time
from collections import deque


def tao_luoi(n, ty_le_tuong, seed):
    while True:
        rng = random.Random(seed)
        luoi = [[1 if rng.random() < ty_le_tuong else 0 for _ in range(n)]
                for _ in range(n)]
        luoi[0][0] = luoi[n - 1][n - 1] = 0
        if tim(luoi, "bfs")[0] is not None:
            return luoi
        seed += 1


def tim(luoi, kieu):
    n = len(luoi)
    dich = (n - 1, n - 1)
    cho = deque([(0, 0)]) if kieu == "bfs" else [(0, 0)]
    cha = {(0, 0): None}
    dinh_cho = 1
    while cho:
        r, c = cho.popleft() if kieu == "bfs" else cho.pop()
        if (r, c) == dich:
            break
        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
```

```
            nr, nc = r + dr, c + dc
            if (0 <= nr < n and 0 <= nc < n and luoi[nr][nc] == 0
                    and (nr, nc) not in cha):
                cha[(nr, nc)] = (r, c)
                cho.append((nr, nc))
        dinh_cho = max(dinh_cho, len(cho))
    if dich not in cha:
        return None, len(cha), dinh_cho
    duong, nut = [], dich
    while nut is not None:
        duong.append(nut)
        nut = cha[nut]
    return duong, len(cha), dinh_cho


def chay(ty_le, kich_thuoc):
    print(f"ty le tuong = {ty_le}")
    print(f"{'kích thước':>10}{'thuật toán':>12}{'ô đã khám phá':>15}"
          f"{'độ dài đường':>14}{'hàng chờ lớn nhất':>19}{'thời gian (ms)':>16}")
    for n in kich_thuoc:
        luoi = tao_luoi(n, ty_le, seed=n)
        for kieu in ["bfs", "dfs"]:
            t = time.perf_counter()
            duong, so_o, dinh = tim(luoi, kieu)
            ms = (time.perf_counter() - t) * 1000
            print(f"{n:>5}x{n:<4}{kieu.upper():>12}{so_o:>15}"
                  f"{len(duong) - 1:>14}{dinh:>19}{ms:>16.2f}")


chay(0.1, [10, 20, 40, 80])
chay(0.3, [10, 20, 40, 80])
chay(0.22, [120])
```

```
chay(0.1, [10, 20, 40, 80])
chay(0.3, [10, 20, 40, 80])
chay(0.22, [120])
```

```
ty le tuong = 0.1
kích thước  thuật toán  ô đã khám phá  độ dài đường  hàng chờ lớn nhất  thời gian (ms)
   10x10          BFS             91            18                 10           0.13
   10x10          DFS             70            36                 34           0.06
   20x20          BFS            353            38                 21           0.47
   20x20          DFS            319           128                150           0.29
   40x40          BFS           1434            78                 42           2.64
   40x40          DFS            886           302                418           0.87
   80x80          BFS           5739           158                 83         100.08
   80x80          DFS           2795           724               1115           3.22
ty le tuong = 0.3
kích thước  thuật toán  ô đã khám phá  độ dài đường  hàng chờ lớn nhất  thời gian (ms)
   10x10          BFS             69            24                  7           0.09
   10x10          DFS             65            24                 11           0.08
   20x20          BFS            279            38                 16           0.36
   20x20          DFS            226            76                 56           0.24
   40x40          BFS           1111            78                 36           1.45
   40x40          DFS            680           278                195           0.71
   80x80          BFS           4405           162                 65           6.45
   80x80          DFS            721           306                236           0.83
ty le tuong = 0.22
kích thước  thuật toán  ô đã khám phá  độ dài đường  hàng chờ lớn nhất  thời gian (ms)
  120x120          BFS          11163           238                121          18.09
  120x120          DFS           1737           678                688           2.26
```

Bài 5 — BFS trên lịch sử Git: tìm commit tổ tiên chung (merge-base)

Liên hệ CNTT: lịch sử Git là một đồ thị có hướng không chu trình (mỗi commit trỏ tới commit cha). Khi hợp nhất hai nhánh, Git cần tìm tổ tiên chung gần nhất (merge-base). Bài này tự cài đặt ý tưởng đó bằng BFS ngược theo quan hệ cha. (Lịch sử giả lập 10 commit.)

📋 Nội dung có sẵn — copy và chạy trên Colab:

```
from collections import deque

# ---- Lịch sử Git: mỗi commit liệt kê commit CHA của nó (merge commit có hai cha)
cha = {
    "c1": [],
    "c2": ["c1"],
    "c3": ["c2"],
    "c4": ["c3"],            # main
    "c5": ["c3"],            # nhánh feature-login bắt đầu từ c3
    "c6": ["c5"],
    "c7": ["c4", "c6"],      # merge feature-login vào main
    "c8": ["c7"],            # main tiếp tục
    "c9": ["c6"],            # nhánh hotfix bắt đầu từ c6
    "c10": ["c9"],
}

def tat_ca_to_tien(cha, commit):
    """BFS ngược theo quan hệ cha; trả về dict {tổ tiên: số bước} (bao gồm chính commit,
khoảng cách 0)"""
    khoang_cach = {commit: 0}
    hang_doi = deque([commit])
    while hang_doi:
        c = hang_doi.popleft()
        for p in cha[c]:
            if p not in khoang_cach:
                khoang_cach[p] = khoang_cach[c] + 1
                hang_doi.append(p)
    return khoang_cach

def merge_base(cha, a, b):
    """Tổ tiên chung 'gần nhất' = tổ tiên chung có tổng khoảng cách tới a và b nhỏ nhất"""
    da, db = tat_ca_to_tien(cha, a), tat_ca_to_tien(cha, b)
    chung = set(da) & set(db)
    return min(chung, key=lambda c: (da[c] + db[c], c)), sorted(chung)
```

```
for a, b in [("c8", "c10"), ("c4", "c6"), ("c10", "c5"), ("c8", "c7")]:
    mb, chung = merge_base(cha, a, b)
    print(f"merge-base({a}, {b}) = {mb}    (các tổ tiên chung: {chung})")

print("\nSố commit mà c8 có thể 'nhìn thấy' (lịch sử đầy đủ):", len(tat_ca_to_tien(cha,
"c8")))
print("Số bước từ c10 về c1:", tat_ca_to_tien(cha, "c10")["c1"])
```

📷 Dán ảnh kết quả merge-base cho các cặp commit

```
def merge_base(cha, a, b):
    """Tổ tiên chung 'gần nhất' = tổ tiên chung có tổng khoảng cách tới a và b nhỏ nhất"""
    da, db = tat_ca_to_tien(cha, a), tat_ca_to_tien(cha, b)
    chung = set(da) & set(db)
    return min(chung, key=lambda c: (da[c] + db[c], c)), sorted(chung)

for a, b in [("c8", "c10"), ("c4", "c6"), ("c10", "c5"), ("c8", "c7")]:
    mb, chung = merge_base(cha, a, b)
    print(f"merge-base({a}, {b}) = {mb}   (các tổ tiên chung: {chung})")

print("\nSố commit mà c8 có thể 'nhìn thấy' (lịch sử đầy đủ):", len(tat_ca_to_tien(cha, "c8")))
print("Số bước từ c10 về c1:", tat_ca_to_tien(cha, "c10")["c1"])
```

```
merge-base(c8, c10) = c6   (các tổ tiên chung: ['c1', 'c2', 'c3', 'c5', 'c6'])
merge-base(c4, c6) = c3   (các tổ tiên chung: ['c1', 'c2', 'c3'])
merge-base(c10, c5) = c5   (các tổ tiên chung: ['c1', 'c2', 'c3', 'c5'])
merge-base(c8, c7) = c7   (các tổ tiên chung: ['c1', 'c2', 'c3', 'c4', 'c5', 'c6', 'c7'])

Số commit mà c8 có thể 'nhìn thấy' (lịch sử đầy đủ): 8
Số bước từ c10 về c1: 6
```

💭 **Suy nghĩ: Vì sao phải duyệt ngược từ commit về các commit cha chứ không duyệt xuôi? merge-base(c10, c5) = c5 nghĩa là gì về quan hệ giữa hai nhánh? Vì sao commit merge (c7) có hai cha và BFS vẫn đúng mà không đếm trùng commit?**

Phải duyệt ngược vì mỗi commit chỉ trỏ tới cha. Từ `c8` muốn biết quá khứ thì đi theo cha. Duyệt xuôi phải dựng thêm đồ thị con, mà dữ liệu không cho sẵn.

`merge-base(c10, c5) = c5` nghĩa là `c5` nằm trên lịch sử của `c10`. Nhánh hotfix tách từ `c6`, và `c6` lại nằm sau `c5`, nên tổ tiên chung gần nhất chính là `c5`.

`c7` có hai cha vì là commit merge. BFS không đếm trùng: commit đã có trong `khoang_cach` thì lần sau bỏ qua, dù đi tới từ cha kia.

🔧 Cải tiến: Thêm commit c11 là merge của c8 và c10 vào từ điển cha, rồi tính merge_base(c11, c5). Sau đó viết hàm commit_chi_co_o(cha, a, b) trả về các commit có trong lịch sử của b mà không có trong lịch sử của a (tương đương git log a..b) và thử với (c8, c10).

```
from collections import deque


cha = {
    "c1": [],
    "c2": ["c1"],
    "c3": ["c2"],
    "c4": ["c3"],
    "c5": ["c3"],
    "c6": ["c5"],
    "c7": ["c4", "c6"],
```

```
    "c8": ["c7"],
    "c9": ["c6"],
    "c10": ["c9"],
    "c11": ["c8", "c10"],   # merge của c8 và c10
}

def tat_ca_to_tien(cha, commit):
    khoang_cach = {commit: 0}
    hang_doi = deque([commit])
    while hang_doi:
        c = hang_doi.popleft()
        for p in cha[c]:
            if p not in khoang_cach:
                khoang_cach[p] = khoang_cach[c] + 1
                hang_doi.append(p)
    return khoang_cach

def merge_base(cha, a, b):
    da, db = tat_ca_to_tien(cha, a), tat_ca_to_tien(cha, b)
    chung = set(da) & set(db)
    return min(chung, key=lambda c: (da[c] + db[c], c)), sorted(chung)

def commit_chi_co_o(cha, a, b):
    """Tương đương git log a..b: commit có trong lịch sử b mà không có trong a."""
    da, db = tat_ca_to_tien(cha, a), tat_ca_to_tien(cha, b)
    return sorted(set(db) - set(da))

mb, chung = merge_base(cha, "c11", "c5")
print(f"merge-base(c11, c5) = {mb}   (các tổ tiên chung: {chung})")
print("git log c8..c10:", commit_chi_co_o(cha, "c8", "c10"))
```

```python
def commit_chi_co_o(cha, a, b):
    """Tương đương git log a..b: commit có trong lịch sử b mà không có trong a."""
    da, db = tat_ca_to_tien(cha, a), tat_ca_to_tien(cha, b)
    return sorted(set(db) - set(da))

mb, chung = merge_base(cha, "c11", "c5")
print(f"merge-base(c11, c5) = {mb}   (các tổ tiên chung: {chung})")
print("git log c8..c10:", commit_chi_co_o(cha, "c8", "c10"))
```

```
merge-base(c11, c5) = c5   (các tổ tiên chung: ['c1', 'c2', 'c3', 'c5'])
git log c8..c10: ['c10', 'c9']
```

BÁO CÁO TRẢI NGHIỆM CUỐI BUỔI

Hoàn thành mục này trước khi xuất PDF và nộp bài:

1. TRẢI NGHIỆM — Tôi đã chạy/thử nghiệm đoạn code/bài toán nào? Kết quả quan sát được là gì?

Mình đã chạy BFS và DFS trên mê cung 5×5, đồ thị các tỉnh, mê cung 8×8, mê cung nhóm 10×10, rồi năm bài mạng nội bộ, crawler, phụ thuộc gói, lưới ngẫu nhiên và lịch sử Git. BFS luôn ra đường ít bước nhất. DFS đôi khi trùng đường nếu thứ tự kề tình cờ đúng, như mê cung gốc và đường HCM–Cần Thơ, nhưng thường dài hơn. Trên mê cung nhóm, BFS đi 9 bước và duyệt 35 ô, DFS đi 45 bước và duyệt 60 ô.

2. QUAN SÁT – PHẢN ÁNH — Kết quả có giống dự đoán ban đầu của tôi không? Điều gì khiến tôi bất ngờ hoặc chưa hiểu rõ?

Lúc đầu mình tưởng hai đường trên mê cung gốc sẽ khác nhau. Chạy ra thì trùng 8 bước, chỉ khác số ô: BFS 16, DFS 10. Đổi goal sang (2,0) thì khác ngay, DFS 14 bước còn BFS 6 bước. Bất ngờ nhất là lưới 80×80: hàng chờ lớn nhất của DFS là 681, của BFS chỉ 82. Mình từng nhớ "DFS tốn ít bộ nhớ", nhưng câu đó không đúng với cách cài stack trên lưới này. Thêm SW1–DB cũng không rút đường xuống dưới 3 bước nhảy, vì đường mới vẫn là GW → SW1 → DB → BACKUP.

3. KHÁI QUÁT HÓA — Không gian trạng thái là gì? BFS và DFS khác nhau như thế nào (cơ chế Queue vs Stack)? (diễn đạt bằng lời của chính mình)

Không gian trạng thái là các tình huống bài toán và các nước đi giữa chúng. Cây tìm kiếm là thứ tự mình mở các tình huống đó từ điểm xuất phát. BFS xếp hàng, lấy người đến trước, nên lan đều theo khoảng cách. DFS xếp chồng, lấy người vừa đến, nên lao sâu một ngả rồi mới quay lại. Chỉ khác chỗ lấy phần tử, nhưng đường tìm được và số ô đã mở vì thế mà khác.

4. ÁP DỤNG – RÚT KINH NGHIỆM — Nếu gặp một bộ dữ liệu mới, tôi sẽ bắt đầu khám phá nó như thế nào? Tôi còn thắc mắc điều gì?

Gặp dữ liệu mới, mình sẽ vẽ lại thành lưới hoặc đồ thị, đếm bậc và tường, rồi hỏi bài toán cần đường ngắn nhất hay chỉ cần một đường. Cần ngắn nhất thì dùng BFS, hoặc Dijkstra nếu có trọng số. Cần thứ tự phụ thuộc hoặc phát hiện vòng thì dùng DFS. Sau đó đo số nút đã mở và frontier, không chỉ nhìn thời gian, vì mê cung nhỏ cho thời gian quá bé. Còn thắc mắc 0-1 BFS đẩy cạnh 0 lên đầu deque khác gì BFS thường, và A* cần heuristic kiểu gì thì vẫn giữ được đường tối ưu.

Mức độ tự tin với chủ đề buổi này (khoanh tròn): 1 — 2 — 3 — 4 — 5

✅ **Sau khi hoàn tất, vào File → Download → PDF Document để xuất tài liệu này (kèm toàn bộ code cải tiến, ảnh và ghi chú của bạn) và nộp theo hướng dẫn của giảng viên.**